INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI

PROJET

PI - SGBD

Étudiants

Hadi	ASSI
Albin	PISSOT
Adam	ARAOUD
Alioune	DIALLO
Noam	PACINI–MARTINS

6 octobre 2026

Table des matières

1 Modélisation et Conception **2**
1.1 Diagramme de Cas d'Utilisation . . . 2
1.2 Modélisation du Diagramme de Classes . . . 2
1.3 Diagramme de Classes Conceptuel . . . 3
1.4 Diagramme de Classes des Exceptions . . . 3

2 Cœur du Moteur **4**
2.1 Implémentation des classes . . . 4
2.1.1 Comparaisons dynamiques et polymorphisme . . . 5
2.2 Tests unitaires . . . 5

3 Interprète (Parseur) **6**
3.1 Fonctions `programme` et `commande` . . . 6
3.2 Fonction `commandeCreate` . . . 6
3.3 Fonction `commandeInsert` . . . 6
3.4 Fonction `commandeSelect` . . . 6
3.5 Fonctions `commandeDelete` et `commandeDrop` . . . 7
3.6 Fonctions `commandeTables` et `commandeExit` . . . 7
3.7 Fonction `source` . . . 7
3.8 Gestion des types, attributs et valeurs . . . 7
3.9 Gestion des conditions . . . 8
3.10 Choix de configuration JavaCC . . . 8

4 Mémoire (Persistance) **9**
4.1 Enregistreur et lecteur . . . 9
4.2 Modification du main . . . 9

5 Conclusion **10**
5.1 Répartition des tâches du projet . . . 10

6 Annexes **11**

1 Modélisation et Conception

Dans un premier temps, nous nous sommes posé différentes questions afin d'avoir une idée plus concrète de ce que nous allions devoir réaliser durant le projet. En effet, nous avons étudié pendant le premier semestre le langage SQL et la gestion de bases de données. Cependant, toute la partie gestion était déjà implémentée et nous n'avions qu'à réaliser des requêtes pour utiliser la base de données. Comme l'enjeu de ce projet est de concevoir notre propre système de gestion de base de données (SGBD), nous devions nous interroger sur la façon de le concevoir. Notre concertation nous a menés aux analyses suivantes :

— **Comment représenter une base de données ?** Il nous faut créer des relations logiques entre les différentes tables qui contiennent les données. Une base de données est essentiellement un *nom* associé à un ensemble de *tables*.

— **Comment représenter une table ? (Quels attributs ? Quelle structure pour stocker les lignes ?)** Une table est une relation entre des données de la base. Elle est représentée par un nom et au minimum un attribut. Un attribut est composé d'un nom et d'un type correspondant à celui des données qu'il contiendra.

1.1 Diagramme de Cas d'Utilisation

Le système propose 8 fonctionnalités. Elles correspondent aux différentes actions que peut réaliser l'utilisateur lorsqu'il souhaite manipuler la base de données. De plus, lorsque l'utilisateur lance le programme, la base de données de la précédente manipulation (si elle existe) est chargée. Elle sera ensuite mise à jour (ou créée si aucune n'est existante) lorsqu'il décidera de quitter le programme. Ces différentes actions sont modélisées dans le diagramme de Cas d'Utilisation (voir figure 1, page 11).

1.2 Modélisation du Diagramme de Classes

Une fois le diagramme de Cas d'Utilisation effectué, il a fallu réfléchir à la manière d'organiser nos différentes classes qui serviront à la gestion d'une base de données. Pour cela, nous avons décidé de répartir les fonctionnalités à implémenter dans différents packages :

- `BaseDeDonnees` : Il s'agit de la classe principale qui permet de gérer les tables de la base. Son rôle est de pouvoir créer et supprimer les tables présentes dans la base de données.
- Package `table` : Dans ce package, nous retrouvons l'ensemble des classes permettant la gestion d'une table et de ses composants. Comme nous l'avions expliqué, une table est constituée d'attributs (les colonnes) caractérisés par le type des données qu'ils vont stocker, et de lignes permettant de lier à chaque attribut une valeur du type correspondant. Afin de gérer la possibilité d'attribuer un alias à une table, nous avons décidé d'ajouter une classe `Alias`. Cette classe nous permettra par la suite de gérer plus facilement les opérations complexes comme l'auto-jointure, qui nécessitera d'effectuer un produit cartésien entre deux mêmes tables et donc de disposer de deux alias différents pour une même classe, contrairement à un simple attribut dans `Table`.
- Package `condition` : C'est ici que toutes les conditions entre les éléments sont gérées. Tout d'abord, l'interface `Condition` est la parente de deux classes : `ConditionSimple` et `ConditionLogique`. Elle permet de définir la méthode *estVerifiee* commune aux deux classes, qui l'implémenteront en fonction du type de condition à vérifier. Dans le cas de `ConditionSimple`, il faut comparer les deux opérandes à l'aide

des opérateurs de l'énumération `Operateur`. Dans le cas de `ConditionLogique`, il faut vérifier si les deux conditions respectent la logique des opérateurs présents dans l'énumération `Logique`. De plus, chacune de ces conditions peut être soit une condition simple, soit une nouvelle condition logique entre deux sous-conditions. Ainsi, grâce à cette organisation, nous pouvons traiter une condition composée de plusieurs sous-conditions.

- Package `valeur` : L'objectif de cet ensemble est de gérer les différents types que peut prendre chaque valeur. Ces types sont au nombre de trois : les chaînes de caractères modélisées par la classe `ValeurVARCHAR`, les entiers par la classe `ValeurINT` et les serials. Ces derniers sont des entiers s'incrémentant automatiquement à chaque insertion. Ils sont modélisés par la classe `ValeurSERIAL`, qui hérite de `ValeurINT` puisqu'un serial se comporte comme un entier, mais constitue un type distinct afin de pouvoir l'identifier facilement et déclencher le mécanisme d'auto-incrémentation lors de l'insertion.

La modélisation du Diagramme de Classes est disponible en figure 2, page 12.

1.3 Diagramme de Classes Conceptuel

L'étape importante de cette partie consistait à adapter notre diagramme de classes de modélisation afin de respecter le cahier des charges nous imposant d'implémenter notre base de données en langage Java. Dans ce diagramme conceptuel, nous avons détaillé les méthodes disponibles pour chaque classe afin de pouvoir implémenter l'interpréteur parallèlement au cœur du moteur.

Tout d'abord, pour gérer la base de données, nous utilisons une `Map` pour stocker les tables. Ainsi, il est impossible d'avoir des tables identiques avec le même nom. De plus, la recherche d'une table parmi toutes celles présentes (dans l'interpréteur par exemple) se fera beaucoup plus rapidement qu'avec une simple liste. Nous évitons ainsi de parcourir toute la liste jusqu'à ce que la table recherchée soit trouvée. Nous utilisons la même logique pour stocker les attributs dans une table, car on ne peut y avoir deux fois le même attribut. Pour gérer l'auto-incrémentation lorsqu'une table est créée avec un attribut de type `SERIAL`, nous avons fait le choix d'utiliser un attribut `serialCounter` qui sera incrémenté lors de l'ajout.

La classe `Attribut` est construite à l'aide d'un **record**, car en dehors des getters et de la méthode `toString()`, aucune autre méthode n'a besoin d'être définie. L'utilisation d'un `record` nous facilite donc l'implémentation de cette classe.

D'autre part, dans la classe `ConditionSimple`, afin de gérer le fait que la partie droite d'une condition puisse être soit un attribut, soit une valeur, nous avons deux attributs distincts `partieDroiteAttribut` et `partieDroiteValeur` prenant chacun le type associé à leur nom. Cette implémentation nous a conduits à définir deux constructeurs.

1.4 Diagramme de Classes des Exceptions

Dans le but de prévoir les cas d'erreurs, mais aussi de bloquer certaines utilisations impossibles de la base de données, nous avons réalisé un diagramme de classes des exceptions (voir figure 4, page 13).

2 Cœur du Moteur

Les diagrammes de classes ayant tous été construits, nous avons pu implémenter la gestion de notre base de données. Pour cela, nous nous sommes répartis les différents packages du projet. Nous avons d'abord commencé par les classes les plus simples et nécessaires pour faire tout le reste, en particulier les classes d'énumérations pour la définition des types.

2.1 Implémentation des classes

L'implémentation a suivi un ordre bottom-up : les classes les plus simples d'abord (**énumérations**, **valeurs**, **attributs**), puis les classes de haut niveau (**Table**, **BaseDeDonnees**) qui en dépendent. Cette approche nous a permis de tester chaque brique indépendamment avant d'assembler le tout.

- La classe centrale du moteur est **Table**. Elle stocke ses attributs dans un **LinkedHashSet** afin de garantir l'ordre d'insertion des colonnes tout en empêchant les doublons. Les lignes sont stockées dans une **List<Ligne>**, chaque `Ligne` étant une **Map<Attribut, Valeur<?>>** associant chaque attribut à sa valeur. Le constructeur de `Ligne` effectue une copie défensive de cette map afin d'en garantir l'immutabilité après construction.
- La méthode **select(Set<Attribut>, Condition, boolean, boolean)** est la méthode centrale pour la consultation. Elle prend en paramètre le sous-ensemble d'attributs à projeter, une condition de filtrage optionnelle, et deux booléens indiquant si la requête porte sur plusieurs tables et si une projection explicite est demandée. Ces informations permettent de nommer automatiquement la table résultante avec le suffixe approprié : `_selection`, `_projection` ou `_selection_projection`. Concrètement, la méthode itère sur toutes les lignes de la table, évalue la condition via `estVerifiee(Ligne)`, et ne conserve que les valeurs des attributs demandés pour construire la table résultat.
- La méthode **produitCartesien(Table)** réalise le produit cartésien de deux tables en combinant chaque ligne de la première avec chaque ligne de la seconde. La table résultante hérite de l'union des attributs des deux tables. C'est cette opération qui sert de base à toutes les jointures : dans la clause FROM avec plusieurs tables, on construit progressivement le produit cartésien, puis la condition WHERE filtre les combinaisons pertinentes, ce qui correspond au comportement attendu de l'algèbre relationnelle.
- La méthode `insert(Map<Attribut, Valeur<?>>)` de la classe `Table` vérifie avant toute insertion que chaque attribut fourni existe dans le schéma de la table et que son type correspond à celui de la valeur associée. Si l'une de ces vérifications échoue, une exception est levée. La gestion du type `SERIAL` est quant à elle déléguée à `BaseDeDonnees` : c'est cette dernière qui injecte automatiquement la valeur du compteur `serialCounter` avant d'appeler `Table.insert()`, ce qui permet de centraliser la logique d'auto-incrémentation indépendamment de la table concernée.

La classe **Alias** joue un rôle crucial pour les jointures et auto-jointures. Sa méthode **getTableAvecPrefixe()** crée une copie de la table où chaque attribut est préfixé par le nom de l'alias (par exemple `heroes.nom` au lieu de `nom`). Cela permet de distinguer les colonnes de même nom provenant de tables différentes lors du produit cartésien et ainsi d'éviter toute ambiguïté lorsque deux tables avec des noms de colonnes similaires sont

traitées.

Enfin, la classe **BaseDeDonnees** stocke les tables dans une **HashMap<String, Table>** pour une recherche insensible à la casse. Elle centralise également la gestion du compteur **serialCounter**, incrémenté globalement à chaque insertion pour attribuer un identifiant unique à tous les attributs de type SERIAL quelle que soit la table concernée.

2.1.1 Comparaisons dynamiques et polymorphisme

Une partie importante de notre projet consistait à permettre la comparaison de données de types différents de manière uniforme. Pour ce faire, nous avons exploité l'interface **Comparable** de Java. Dans la méthode `estVerifiee` de la classe **ConditionSimple**, les valeurs extraites des lignes sont castées en **Comparable**. Cette approche permet à notre moteur d'utiliser les mêmes opérateurs (EGAL, SUPERIEUR, INFERIEUR) sans distinction de type, tout en nous assurant que les comparaisons respectent l'ordre au sein du type.

2.2 Tests unitaires

Afin de tester notre projet, nous avons implémenté des tests unitaires pour chaque classe. Pour cela, chacun de nous a réalisé les tests sur une partie qu'il n'avait pas codée dans la phase précédente. Cela nous a permis de vérifier comment les méthodes devaient réellement fonctionner, indépendamment de l'implémentation en elle-même. Ainsi, nous avons pu détecter les erreurs commises et les corriger au fur et à mesure avec plus de facilité que si la même personne avait réalisé ses propres tests. En effet, une vue extérieure à celui qui a implémenté les méthodes permet d'apporter une vision différente et ainsi de s'apercevoir des potentielles erreurs commises. De plus, lors de la réalisation des tests, nous avons pu réfléchir à des exceptions pouvant survenir auxquelles nous n'avions pas pensé durant notre phase de réflexion lors de la conception des diagrammes de classes.

3 Interprète (Parseur)

Le fichier `AnalyseurSyntaxique.jj` contient la grammaire JavaCC permettant d'analyser les commandes SQL saisies par l'utilisateur. Nous avons choisi de découper cette grammaire en plusieurs fonctions spécialisées afin d'obtenir un parseur plus lisible, plus modulaire et plus facile à maintenir. Chaque fonction analyse une partie précise du langage et construit directement les objets Java utilisés par le moteur de base de données.

3.1 Fonctions `programme` et `commande`

La fonction `programme` constitue le point d'entrée du parseur. Elle initialise la session utilisateur puis lit successivement les commandes SQL jusqu'à la fin de l'entrée. Ce choix permet à l'utilisateur d'exécuter plusieurs requêtes dans une même session, comme dans un véritable SGBD interactif.

La fonction `commande` sert ensuite de point de répartition entre les différentes commandes supportées : `CREATE TABLE`, `INSERT INTO`, `SELECT`, `DROP TABLE`, `DELETE FROM`, `TABLES` et `EXIT`. Chaque commande est traitée dans une fonction dédiée, ce qui évite d'avoir une méthode unique trop complexe. L'utilisation obligatoire du point-virgule en fin de commande permet également de délimiter clairement les requêtes et de détecter plus facilement les erreurs de syntaxe.

3.2 Fonction `commandeCreate`

La fonction `commandeCreate` est responsable de l'analyse des commandes de création de table. Elle récupère le nom de la table ainsi que la liste des attributs qui composent son schéma, puis délègue la création effective à la classe représentant la base de données.

Nous avons choisi de construire le schéma dès l'analyse syntaxique afin de vérifier rapidement la cohérence des informations fournies. Cette fonction permet notamment de détecter les tentatives de création d'une table déjà existante et d'afficher un message d'erreur explicite. Elle participe donc à la validation de la structure de la base avant toute modification de son contenu.

3.3 Fonction `commandeInsert`

La fonction `commandeInsert` traite l'insertion de données dans une table. Elle analyse d'abord la liste des colonnes indiquées par l'utilisateur, puis la liste des valeurs à insérer. Ces deux listes sont ensuite associées à l'aide d'une structure de type `Map<Attribut, Valeur<?>>`.

Ce choix permet de relier explicitement chaque valeur à son attribut correspondant, au lieu de dépendre uniquement de l'ordre des éléments dans la requête. Avant l'insertion, plusieurs vérifications sont réalisées : existence de la table, correspondance entre le nombre de colonnes et le nombre de valeurs, puis compatibilité des types. Ces contrôles permettent de préserver l'intégrité des données stockées dans la base.

3.4 Fonction `commandeSelect`

La fonction `commandeSelect` est la plus importante du parseur, car elle gère les requêtes de consultation. Elle analyse les colonnes demandées, les tables présentes dans la

clause `FROM`, les alias éventuels, la clause `WHERE`, puis applique la projection finale des attributs sélectionnés.

Lorsque la requête contient une seule table, celle-ci est utilisée directement. En revanche, lorsque plusieurs tables sont présentes, la fonction construit progressivement un produit cartésien entre les tables préfixées par leur nom ou leur alias. Les conditions de la clause `WHERE` sont ensuite appliquées sur cette table intermédiaire.

Ce choix suit les principes de l'algèbre relationnelle : une jointure peut être vue comme une sélection appliquée sur un produit cartésien. Cette approche simplifie l'implémentation des jointures tout en conservant un comportement proche d'un SGBD classique.

3.5 Fonctions `commandeDelete` et `commandeDrop`

La fonction `commandeDelete` permet de supprimer des lignes dans une table. Si aucune clause `WHERE` n'est fournie, toutes les lignes de la table sont supprimées. Sinon, seules les lignes respectant la condition sont retirées. Ce comportement correspond à celui des systèmes SQL classiques.

La fonction `commandeDrop`, quant à elle, permet de supprimer entièrement une table de la base. Cette opération est déléguée à la classe représentant la base de données afin de centraliser les modifications de structure. La fonction vérifie aussi l'existence de la table avant la suppression, ce qui évite les incohérences.

3.6 Fonctions `commandeTables` et `commandeExit`

La fonction `commandeTables` affiche la liste des tables actuellement présentes dans la base. Elle a été ajoutée pour améliorer l'ergonomie du programme et faciliter les tests pendant le développement.

La fonction `commandeExit` permet de quitter proprement la session. Nous avons choisi d'utiliser une exception dédiée, `ExitException`, plutôt qu'une variable booléenne partagée. Cette solution interrompt directement le flux d'exécution tout en gardant un code simple et lisible.

3.7 Fonction `source`

La fonction `source` est utilisée dans les requêtes `SELECT` pour analyser les tables présentes dans la clause `FROM`. Elle récupère la table demandée et construit un objet `Alias` contenant la table ainsi que son alias éventuel.

La gestion des alias a été intégrée afin de simplifier les requêtes impliquant plusieurs tables. Elle permet aussi de gérer les auto-jointures, où une même table peut apparaître plusieurs fois dans une requête. Si aucun alias n'est précisé, le nom de la table est utilisé par défaut. La fonction vérifie également l'existence de la table dès l'analyse de la requête afin de signaler rapidement les erreurs.

3.8 Gestion des types, attributs et valeurs

Les fonctions `type`, `attribut`, `listeAttributs` et `valeur` permettent de convertir les éléments lus dans la requête SQL en objets Java exploitables par le moteur.

La fonction `type` transforme les types SQL `INT`, `SERIAL` et `VARCHAR` en valeurs de l'énumération `TypeAttribut`. La fonction `attribut` construit ensuite un objet `Attribut` à partir d'un nom et d'un type. La fonction `listeAttributs` regroupe l'ensemble des

attributs dans un `Set`, ce qui permet d'éviter automatiquement les doublons dans le schéma d'une table.

La fonction `valeur` crée les objets correspondant aux valeurs manipulées dans les requêtes. Un nombre est transformé en `ValeurINT`, tandis qu'une chaîne de caractères est transformée en `ValeurVARCHAR`. Cette conversion dès l'analyse syntaxique permet au reste du programme de manipuler des objets typés plutôt que de simples chaînes de caractères.

3.9 Gestion des conditions

Les fonctions `conditionSimple` et `condition` permettent d'analyser les clauses `WHERE`.

La fonction `conditionSimple` construit une condition élémentaire à partir d'un attribut, d'un opérateur de comparaison et d'une valeur ou d'un autre attribut. Elle permet donc de gérer aussi bien les comparaisons classiques, comme `age > 18`, que les comparaisons entre colonnes, utiles pour les jointures, comme `Hero.id = Pouvoir.heroId`.

La fonction vérifie également l'existence des attributs référencés dans la table courante. Les erreurs liées aux noms de colonnes sont ainsi détectées avant l'exécution de la requête.

La fonction `condition` permet ensuite de combiner plusieurs conditions simples avec les opérateurs logiques `AND` et `OR`. À chaque opérateur rencontré, un objet `ConditionLogique` est créé. Cette construction progressive produit une représentation logique directement exploitable lors des opérations de sélection ou de suppression.

3.10 Choix de configuration JavaCC

Plusieurs options JavaCC ont été utilisées pour améliorer le comportement du parseur. L'option `IGNORE_CASE=true` rend les commandes SQL insensibles à la casse, ce qui permet d'accepter aussi bien `select`, `SELECT` ou `Select`.

L'option `STATIC=false` permet de créer des instances non statiques du parseur et facilite sa réinitialisation avec `ReInit`. Cela évite de recréer entièrement un analyseur à chaque commande.

Enfin, les options liées à la détection d'ambiguïtés, comme `CHOICE_AMBIGUITY_CHECK`, `OTHER_AMBIGUITY_CHECK` et `FORCE_LA_CHECK`, ont été conservées afin d'aider à repérer les ambiguïtés potentielles dans la grammaire pendant le développement.

4 Mémoire (Persistance)

L'objectif de cette partie est de pouvoir automatiquement charger et sauvegarder la base de données respectivement en début et en fin de programme. Comme l'indiquait le sujet, nous avons choisi de réaliser une implémentation simple en écrivant ou en lisant l'objet instance de `BaseDeDonnees` dans un flux, c'est-à-dire par sérialisation.

4.1 Enregistreur et lecteur

Pour la classe `Enregistreur`, nous avons défini un attribut de type `ObjectOutputStream` afin d'établir le flux d'enregistrement à l'instanciation. Nous avons ensuite implémenté une méthode `enregistrer` permettant de prendre un objet `BaseDeDonnees` et de l'écrire dans le flux. À l'inverse, la classe `Lecteur` contient un attribut de type `BaseDeDonnees` qui est lu depuis un flux passé en paramètre lors de l'instanciation, et un simple getter permet d'obtenir l'objet `BaseDeDonnees`.

4.2 Modification du main

Dans la classe `Main`, nous avons ajouté deux méthodes : `charger` et `sauvegarder`. Ces méthodes instancient respectivement des objets `Lecteur` et `Enregistreur` avec le type de flux correspondant, instancié à partir d'un fichier nommé « data », puis appellent les méthodes correspondantes. Ces méthodes sont appelées respectivement au début et à la fin du programme principal. Afin de gérer le cas où le fichier « data » est absent ou vide, nous vérifions, suite à l'appel de la méthode `charger`, si une base de données a bien été instanciée. Si ce n'est pas le cas, nous procédons alors à son instanciation.

5 Conclusion

L'objectif de ce projet était de concevoir notre propre système de gestion de base de données en Java. Pour cela, nous avons dû mettre en application les connaissances que nous avons pu acquérir à travers les matières de `Programmation Avancée`, `Compilation` et `UML`. Durant toute la réalisation du projet, notre démarche est restée guidée par la même question qu'au début, c'est à dire :
Comment lier efficacement toutes les parties du projet ? Le plus grand défi a été la communication entre le cœur du moteur et l'interpréteur JavaCC. En effet, nous avons pu nous apercevoir qu'un changement de méthode ou de type dans un package pouvait avoir un impact sur l'écriture des fonctions du parseur. Il a donc fallut mettre à jour les diagrammes ainsi qu'une bonne communication entre les deux parties afin de pouvoir continuer à avancer parallèlement et toujours dans la même direction.

Au final, nous avons réussi à implémenter un système fonctionnel qui respecte le cahier des charges. Pour pousser le projet encore plus loin, nous pourrions envisager d'inclure les contraintes d'intégrités lors de la création des tables. Ce projet aura été une expérience enrichissante pour nous initier à la conception d'un logicielle en équipe.

5.1 Répartition des tâches du projet

Tâches / Étudiants	**Hadi**	**Albin**	**Adam**	**Alioune**	**Noam**
PARTIE 1 : Modélisation et Conception					
Diagrammes (Cas d'utilisation / Classes)	X	X	X	X	X
PARTIE 2 : Cœur du Moteur					
Gestion de la base de données					X
Gestion des conditions					X
Gestion de la table		X		X	
Gestion des valeurs		X			
Implémentation des tests		X		X	X
PARTIE 3 : Interprète (Parseur SQL)					
Grammaire JavaCC (.jj)	X		X		
Commandes (CREATE, INSERT, SELECT...)	X		X		
Gestion des Conditions et Filtres	X				
PARTIE 4 : Mémoire (Persistance)					
Sérialisation (Lecteur / Enregistreur)					X
Rédaction du rapport final	X	X	X	X	X

Table 1 – Répartition détaillée des tâches par section de projet

6 Annexes

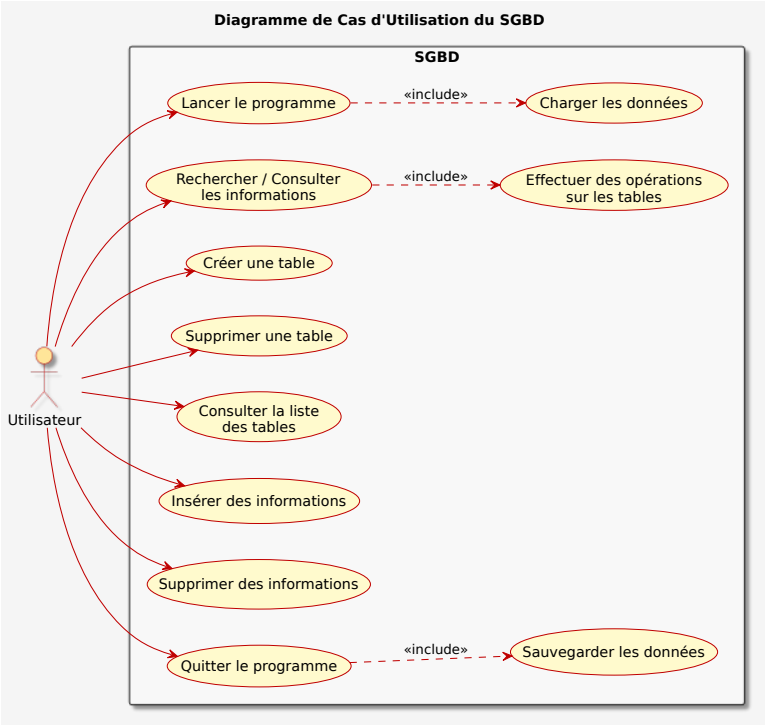


FIGURE 1 – Diagramme de Cas d'Utilisation du SGBD

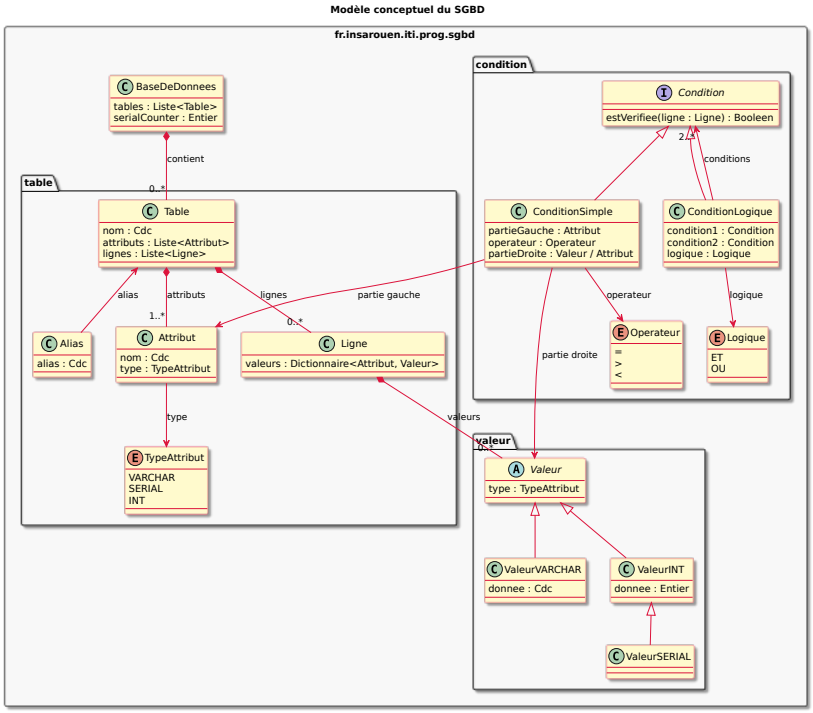


FIGURE 2 – Diagramme de Classes modélisation

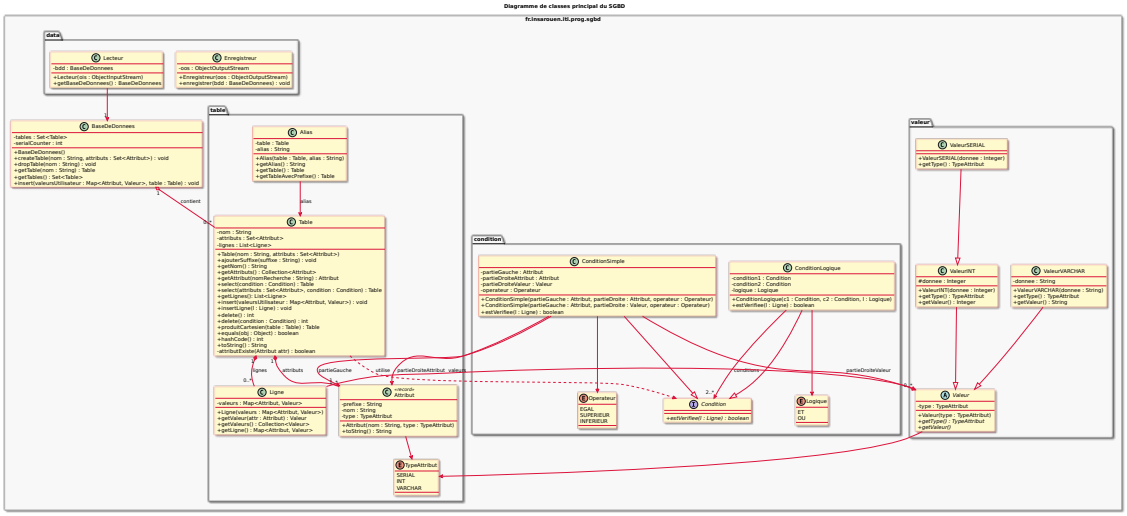


FIGURE 3 – Diagramme de Classes conception (adapté au langage java)

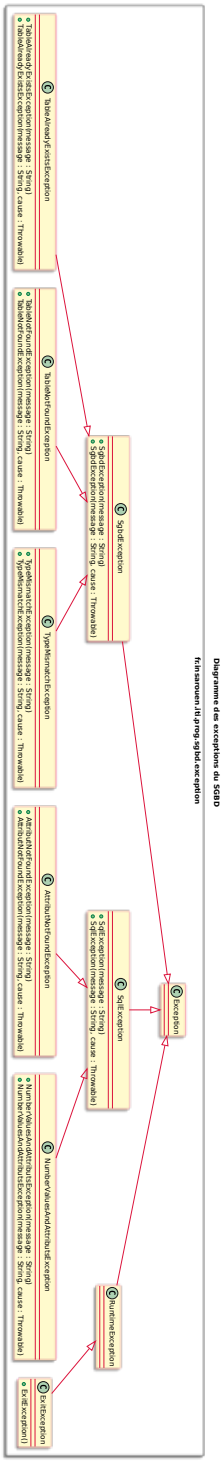


FIGURE 4 – Diagramme de Classes des Exceptions